1. University Scholarship Allocation

Problem statement: Check applicants against aggregate marks, subject-wise minimum marks, attendance, and family income. List eligible applicants by rank; break ties using higher aggregate, then higher attendance.

Algorithm

1. Start and read the number of applicants.
2. For each applicant, read name, aggregate, subject marks, attendance, and annual family income.
3. Mark eligible only if aggregate ≥ 75, every subject mark ≥ 60, attendance ≥ 85%, and income ≤ ■2,50,000.
4. Store eligible applicants and sort by aggregate (descending), then attendance (descending).
5. Display the ranked eligible list; if none qualify, display a message.
6. Stop.

Flowchart (text form)

Start → Read applicants → Check all four eligibility conditions → Eligible? → Yes: store applicant → No: skip applicant → Repeat for all → Sort eligible applicants by marks, then attendance → Display ranking → Stop

Pseudocode

```
BEGIN
  eligible = empty list
  FOR each applicant
    READ name, aggregate, subject_marks, attendance, income
    IF aggregate >= 75 AND MIN(subject_marks) >= 60
       AND attendance >= 85 AND income <= 250000 THEN
       ADD applicant to eligible
    END IF
  END FOR
  SORT eligible by aggregate descending, attendance descending
  DISPLAY eligible ranking
END
```

Python code

```
applicants = [
    ("Anu", 88, [80, 76, 91], 92, 180000),
    ("Balu", 82, [65, 70, 60], 88, 220000),
    ("Chitra", 91, [90, 55, 95], 96, 150000),
    ("Deepak", 88, [75, 80, 85], 92, 200000)
]

eligible = []
for name, aggregate, subjects, attendance, income in applicants:
    if (aggregate >= 75 and min(subjects) >= 60
            and attendance >= 85 and income <= 250000):
        eligible.append((name, aggregate, attendance))

eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("Scholarship ranking")
if not eligible:
    print("No eligible applicants")
else:
    for rank, student in enumerate(eligible, 1):
        print(rank, student[0], "Marks:", student[1],
              "Attendance:", str(student[2]) + "%")
```

Sample output

```
Scholarship ranking
1 Anu Marks: 88 Attendance: 92%
2 Deepak Marks: 88 Attendance: 92%
3 Balu Marks: 82 Attendance: 88%
```

Note: The example uses sample policy limits. Anu and Deepak tie on both criteria, so their original input order is retained.

2. Progressive Electricity Billing

Problem statement: Calculate monthly electricity bills using progressive consumption slabs, a fixed charge, surcharge, and rebate. Compare bills and flag an increase greater than 30%.

Algorithm

1. Read units consumed for each month.

2. Charge ■2.00 per unit for the first 100 units, ■3.00 for the next 100, and ■5.00 for units above 200.

3. Add a fixed charge of ■50.

4. If consumption is below 100 units, apply a 5% rebate to the energy charge; otherwise add a 5% surcharge to the energy charge.

5. Store and display each bill. Compare each month with the previous month and flag increases above 30%.

6. Stop.

Flowchart (text form)

Start → Read monthly units → Calculate slab-wise energy charge → Apply rebate or surcharge → Add fixed charge → Store bill → More months? → Yes: repeat → Compare month-to-month bills → Flag increases > 30% → Stop

Pseudocode

```
BEGIN
  FOR each month
    READ units
    CALCULATE energy charge using progressive slabs
    IF units < 100 THEN energy = energy - 5% of energy
    ELSE energy = energy + 5% of energy
    bill = energy + 50
    STORE bill
  END FOR
  DISPLAY bills
  COMPARE each bill with previous bill
  IF increase percentage > 30 THEN DISPLAY unusual increase
END
```

Python code

```
units_by_month = [80, 120, 260]
bills = []

for month, units in enumerate(units_by_month, 1):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    if units < 100:
        energy *= 0.95
    else:
        energy *= 1.05

    bill = energy + 50
    bills.append(bill)
    print("Month", month, "Units:", units, "Bill: Rs.", round(bill, 2))

for i in range(1, len(bills)):
    increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
    if increase > 30:
        print("Unusual increase in month", i + 1,
              ":", round(increase, 1), "%")
```

Sample output

```
Month 1 Units: 80 Bill: Rs. 202.0
Month 2 Units: 120 Bill: Rs. 365.0
Month 3 Units: 260 Bill: Rs. 1080.0
Unusual increase in month 2 : 80.7 %
Unusual increase in month 3 : 195.9 %
```

Note: The slabs are progressive: only the units inside each slab receive that slab's rate.

3. Hospital Billing and Patient Classification

Problem statement: Add consultation, diagnostic, and treatment charges; apply a concession for senior citizens or low-income patients; validate the bill and classify payment status.

Algorithm

1. Read patient name, age, income, consultation, diagnostic, treatment, and amount paid.
2. Check that all charges and payment are non-negative.
3. Calculate subtotal = consultation + diagnostics + treatment.
4. Apply 10% concession if age is at least 60 or annual income is below ■1,50,000.
5. Calculate final bill and balance. Classify as Paid, Partly Paid, or Unpaid.
6. Display itemized bill and classification.

Flowchart (text form)

Start → Read patient and charges → Validate non-negative values → Invalid? → Yes: show error → No: total charges → Check concession condition → Calculate final bill and balance → Classify payment → Print bill → Stop

Pseudocode

```
BEGIN
  READ patient details and charges
  IF any charge < 0 OR paid < 0 THEN DISPLAY error
  ELSE
    subtotal = consultation + diagnostics + treatment
    IF age >= 60 OR income < 150000 THEN discount = 10% of subtotal
    ELSE discount = 0
    final_bill = subtotal - discount
    balance = final_bill - paid
    IF balance <= 0: status = Paid
    ELSE IF paid > 0: status = Partly Paid
    ELSE: status = Unpaid
    DISPLAY bill and status
  END IF
END
```

Python code

```
name = "Ravi"
age = 65
income = 180000
consultation = 500
diagnostics = 1200
treatment = 3300
paid = 3000

if min(consultation, diagnostics, treatment, paid) < 0:
    print("Error: amounts cannot be negative")
else:
    subtotal = consultation + diagnostics + treatment
    discount = subtotal * 0.10 if age >= 60 or income < 150000 else 0
    final_bill = subtotal - discount
    balance = max(0, final_bill - paid)

    if balance == 0:
        status = "Paid"
    elif paid > 0:
        status = "Partly Paid"
    else:
        status = "Unpaid"

    print("Patient:", name)
    print("Subtotal: Rs.", subtotal)
    print("Concession: Rs.", round(discount, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Paid: Rs.", paid, "Balance: Rs.", round(balance, 2))
    print("Classification:", status)
```

Sample output

```
Patient: Ravi
```

```
Subtotal: Rs. 5000
Concession: Rs. 500.0
Final bill: Rs. 4500.0
Paid: Rs. 3000 Balance: Rs. 1500.0
Classification: Partly Paid
```

Note: This is a classroom example, not a real hospital's billing policy.

4. Employee Payroll and Deduction Analysis

Problem statement: Calculate gross salary from basic pay, allowances, overtime, and incentive. Apply progressive deductions and print a salary statement.

Algorithm

1. Read employee name, basic pay, allowances, overtime pay, and incentive.
2. Calculate gross salary as the sum of these amounts.
3. Apply deductions: 5% on the first ■20,000, 10% on the next ■20,000, and 15% on the amount above ■40,000.
4. Calculate net salary = gross salary – deduction.
5. Display a formatted salary statement.

Flowchart (text form)

Start → Read salary components → Validate amounts → Calculate gross pay → Calculate deductions by slabs → Calculate net salary → Print salary statement → Stop

Pseudocode

```
BEGIN
  READ basic, allowances, overtime, incentive
  gross = basic + allowances + overtime + incentive
  deduction = 5% of first 20000
  + 10% of next 20000
  + 15% of amount above 40000
  net = gross - deduction
  DISPLAY gross, deduction, net
END
```

Python code

```
name = "Meena"
basic = 30000
allowances = 8000
overtime = 3000
incentive = 2000

gross = basic + allowances + overtime + incentive

if gross <= 20000:
    deduction = gross * 0.05
elif gross <= 40000:
    deduction = 20000 * 0.05 + (gross - 20000) * 0.10
else:
    deduction = (20000 * 0.05 + 20000 * 0.10
                 + (gross - 40000) * 0.15)

net = gross - deduction
print("SALARY STATEMENT")
print("Employee:", name)
print("Gross salary: Rs.", round(gross, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net, 2))
```

Sample output

```
SALARY STATEMENT
Employee: Meena
Gross salary: Rs. 43000
Deduction: Rs. 3450.0
Net salary: Rs. 39550.0
```

Note: The deduction rates are illustrative. Each rate applies only to the part of salary in its slab.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement: Compare mobile plans using expected data usage, validity, and budget. Calculate excess-data charges and choose the cheapest eligible plan.

Algorithm

1. Read expected data usage, required validity, and budget.
2. For each plan, check whether validity meets the requirement.
3. Calculate extra data beyond the included allowance and multiply by the plan's excess rate.
4. Calculate total cost = plan price + excess charges.
5. Keep plans whose total cost is within budget and select the lowest-cost plan.
6. Display the selected plan or report that no plan meets the requirements.

Flowchart (text form)

Start → Read usage, validity, budget → Evaluate each plan → Validity sufficient? → Calculate excess charges and total cost → Within budget? → Keep eligible plan → More plans? → Select minimum cost → Display result → Stop

Pseudocode

```
BEGIN
  READ usage, required_validity, budget
  best_plan = NONE
  FOR each plan
    IF plan.validity >= required_validity THEN
      extra = MAX(0, usage - plan.included_data)
      cost = plan.price + extra * plan.extra_rate
      IF cost <= budget AND (best_plan is NONE OR cost < best_cost)
        best_plan = plan
        best_cost = cost
      END IF
    END IF
  END FOR
  DISPLAY best_plan or no suitable plan
END
```

Python code

```
usage_gb = 18
required_days = 28
budget = 400
# Plan: name, price, included GB, extra charge per GB, validity days
plans = [
    ("Basic", 199, 10, 20, 14),
    ("Smart", 299, 20, 5, 28),
    ("Max", 399, 30, 0, 28)
]
best = None

for name, price, included_gb, extra_rate, validity in plans:
    if validity >= required_days:
        extra = max(0, usage_gb - included_gb)
        cost = price + extra * extra_rate
        if cost <= budget and (best is None or cost < best[1]):
            best = (name, cost)

if best:
    print("Best plan:", best[0])
    print("Estimated cost: Rs.", best[1])
else:
    print("No plan meets the budget and validity requirements")
```

Sample output

```
Best plan: Smart
Estimated cost: Rs. 299
```

Note: Plan values are illustrative. A plan must meet both the validity and budget conditions.

6. Railway Fare and Reservation Calculator

Problem statement: Calculate fares for passengers based on distance slabs, class multiplier, age concession, reservation fee, and cancellation penalty.

Algorithm

1. Read passenger details, distance, class, age, and booking/cancellation choice.
2. Calculate base fare: ■1.20 per km for first 200 km, ■1.00 per km for next 300 km, and ■0.80 per km beyond 500 km.
3. Multiply by class factor: Sleeper 1.0, AC 2.0.
4. Apply a 50% concession for passengers aged 60 or above.
5. Add ■40 reservation fee for a booking; if cancelled, deduct a 25% cancellation penalty from the fare and show refund.

Flowchart (text form)

Start → Read trip and passenger details → Calculate distance-slab fare → Apply class multiplier → Apply age concession → Booking or cancellation? → Add reservation fee OR calculate penalty/refund → Display amount → Stop

Pseudocode

```
BEGIN
  READ distance, class, age, action
  CALCULATE base fare using distance slabs
  IF class = AC THEN fare = base * 2 ELSE fare = base
  IF age >= 60 THEN fare = fare * 0.50
  IF action = booking THEN total = fare + 40
  ELSE penalty = fare * 0.25; refund = fare - penalty
  DISPLAY total or refund
END
```

Python code

```
distance = 450
travel_class = "AC"
age = 65
action = "book"

if distance <= 200:
    base = distance * 1.20
elif distance <= 500:
    base = 200 * 1.20 + (distance - 200) * 1.00
else:
    base = 200 * 1.20 + 300 * 1.00 + (distance - 500) * 0.80

fare = base * (2 if travel_class == "AC" else 1)
if age >= 60:
    fare *= 0.50

if action == "book":
    print("Fare after concession: Rs.", round(fare, 2))
    print("Reservation fee: Rs. 40")
    print("Total payable: Rs.", round(fare + 40, 2))
else:
    penalty = fare * 0.25
    print("Cancellation penalty: Rs.", round(penalty, 2))
    print("Refund: Rs.", round(fare - penalty, 2))
```

Sample output

```
Fare after concession: Rs. 310.0
Reservation fee: Rs. 40
Total payable: Rs. 350.0
```

Note: Fare rates and concessions are sample values for programming practice, not actual railway tariffs.

7. Retail Billing with Multiple Discount Rules

Problem statement: Create an itemized invoice for products with quantities, category taxes, a threshold discount, and a maximum concession limit.

Algorithm

1. Read each product's name, category, unit price, and quantity.
2. Calculate line total = unit price × quantity; add line totals to subtotal.
3. Apply a 10% discount if subtotal is at least ■5,000, capped at ■800.
4. Calculate tax on the discounted amount: electronics 18%, clothing 5%, other items 12%.
5. Calculate final amount = discounted subtotal + tax and print the invoice.

Flowchart (text form)

Start → Read products → Calculate line totals and subtotal → Check discount threshold → Apply discount with cap → Calculate category taxes → Add taxes → Print itemized invoice → Stop

Pseudocode

```
BEGIN
  subtotal = 0
  FOR each product
    line_total = price * quantity
    subtotal = subtotal + line_total
  END FOR
  IF subtotal >= 5000 THEN discount = MIN(subtotal * 10%, 800)
  ELSE discount = 0
  discounted = subtotal - discount
  CALCULATE tax by product category
  total = discounted + total_tax
  DISPLAY invoice
END
```

Python code

```
items = [
    ("Headphones", "electronics", 3000, 1),
    ("Shirt", "clothing", 1200, 2)
]
subtotal = 0
for name, category, price, qty in items:
    line = price * qty
    subtotal += line
    print(name, "Qty:", qty, "Line total: Rs.", line)

discount = min(subtotal * 0.10, 800) if subtotal >= 5000 else 0
discounted_total = subtotal - discount
tax = 0
for name, category, price, qty in items:
    rate = 0.18 if category == "electronics" else (
        0.05 if category == "clothing" else 0.12)
    tax += price * qty * rate

print("Subtotal: Rs.", subtotal)
print("Discount: Rs.", round(discount, 2))
print("Tax: Rs.", round(tax, 2))
print("Final invoice: Rs.", round(discounted_total + tax, 2))
```

Sample output

```
Headphones Qty: 1 Line total: Rs. 3000
Shirt Qty: 2 Line total: Rs. 2400
Subtotal: Rs. 5400
Discount: Rs. 540.0
Tax: Rs. 660.0
Final invoice: Rs. 5520.0
```

Note: This example computes category tax on original line totals, then subtracts the threshold discount from the subtotal. A real tax system may require a different tax base.

8. Manufacturing Cost and Profitability Analysis

Problem statement: Calculate material, labour, wastage, and machine costs for a batch. Compare profit per unit with a target.

Algorithm

1. Read quantity, material cost per unit, labour cost per unit, wastage percentage, machine cost, and selling price per unit.

2. Calculate material cost and wastage cost.

3. Calculate total production cost = material + wastage + labour + machine cost.

4. Calculate revenue = quantity × selling price; profit = revenue – total cost.

5. Calculate profit per unit and report whether the target is met.

Flowchart (text form)

Start → Read production inputs → Calculate material and wastage costs → Add labour and machine costs → Calculate revenue and profit → Compare profit per unit with target → Display result → Stop

Pseudocode

```
BEGIN
  READ quantity, material/unit, labour/unit, wastage%, machine, selling price, target
  material_total = quantity * material/unit
  wastage = material_total * wastage% / 100
  total_cost = material_total + wastage + quantity * labour/unit + machine
  revenue = quantity * selling price
  profit = revenue - total_cost
  profit_per_unit = profit / quantity
  IF profit_per_unit >= target THEN DISPLAY target met
  ELSE DISPLAY target not met
END
```

Python code

```
quantity = 100
material_per_unit = 120
labour_per_unit = 40
wastage_percent = 5
machine_cost = 2000
selling_price = 220
target_profit_per_unit = 30

material = quantity * material_per_unit
wastage = material * wastage_percent / 100
labour = quantity * labour_per_unit
total_cost = material + wastage + labour + machine_cost
revenue = quantity * selling_price
profit = revenue - total_cost
per_unit = profit / quantity

print("Total production cost: Rs.", round(total_cost, 2))
print("Revenue: Rs.", round(revenue, 2))
print("Profit: Rs.", round(profit, 2))
print("Profit per unit: Rs.", round(per_unit, 2))
print("Target met:", "Yes" if per_unit >= target_profit_per_unit else "No")
```

Sample output

```
Total production cost: Rs. 18800.0
Revenue: Rs. 22000
Profit: Rs. 3200.0
Profit per unit: Rs. 32.0
Target met: Yes
```

Note: Wastage is modelled as a percentage of material cost.

9. Multi-Currency Travel Budget Analysis

Problem statement: Convert expenses in foreign currencies using supplied exchange rates, add conversion fees, compare spending with budget, and identify the largest category.

Algorithm

1. Read the home-currency budget and each expense category, foreign amount, and exchange rate.
2. Convert each amount into rupees using amount × exchange rate.
3. Add a 2% conversion fee to each converted expense.
4. Calculate total spending and compare it with the budget.
5. Identify the category with the largest rupee cost and display remaining balance or overspending.

Flowchart (text form)

Start → Read budget and expenses → Convert each expense → Add conversion fee → Store category totals → Sum spending → Find largest category → Compare with budget → Display report → Stop

Pseudocode

```
BEGIN
  total = 0
  FOR each expense
    rupees = foreign_amount * exchange_rate
    fee = rupees * 2%
    category_total = rupees + fee
    total = total + category_total
  END FOR
  largest = category with maximum category_total
  IF total <= budget THEN remaining = budget - total
  ELSE overspend = total - budget
  DISPLAY total, largest, remaining or overspend
END
```

Python code

```
budget = 50000
expenses = [
    ("Hotel", 200, 83.0),
    ("Food", 100, 83.0),
    ("Transport", 80, 83.0)
]
converted = []
total = 0

for category, amount, rate in expenses:
    rupees = amount * rate
    fee = rupees * 0.02
    final = rupees + fee
    converted.append((category, final))
    total += final
    print(category, ": Rs.", round(final, 2))

largest = max(converted, key=lambda x: x[1])
print("Total spending: Rs.", round(total, 2))
print("Largest category:", largest[0])
if total <= budget:
    print("Budget remaining: Rs.", round(budget - total, 2))
else:
    print("Over budget by: Rs.", round(total - budget, 2))
```

Sample output

```
Hotel : Rs. 16932.0
Food : Rs. 8466.0
Transport : Rs. 6772.8
Total spending: Rs. 32170.8
Largest category: Hotel
Budget remaining: Rs. 17829.2
```

Note: Exchange rates are supplied sample values and should be updated for a real trip.

10. Campus Event Financial Planning

Problem statement: Calculate event costs, subtract sponsorship, calculate participant contributions, and find the minimum registration count needed to break even.

Algorithm

1. Read venue, equipment, catering per participant, transport per participant, certificate per participant, and sponsorship.

2. Calculate fixed cost = venue + equipment.

3. Calculate variable cost per participant = catering + transport + certificate.

4. Calculate break-even registrations as the ceiling of (fixed cost – sponsorship) divided by (registration fee – variable cost per participant), if the denominator is positive.

5. Calculate expected total cost and display sponsorship shortfall or surplus.

Flowchart (text form)

Start → Read event costs, sponsorship, fee and expected registrations → Calculate fixed and variable costs → Calculate net fixed cost → Is fee > variable cost? → No: break-even impossible → Yes: calculate minimum registrations → Calculate expected budget → Display report → Stop

Pseudocode

```
BEGIN
  fixed = venue + equipment
  variable_per_person = catering + transport + certificate
  net_fixed = MAX(0, fixed - sponsorship)
  contribution = registration_fee - variable_per_person
  IF contribution <= 0 THEN break_even = impossible
  ELSE break_even = CEILING(net_fixed / contribution)
  expected_cost = fixed + expected_people * variable_per_person
  expected_income = sponsorship + expected_people * registration_fee
  DISPLAY break_even, expected_cost, expected_income, shortfall/surplus
END
```

Python code

```
venue = 12000
equipment = 5000
catering_per_person = 180
transport_per_person = 50
certificate_per_person = 20
sponsorship = 10000
registration_fee = 300
expected_people = 100

fixed = venue + equipment
variable = catering_per_person + transport_per_person + certificate_per_person
contribution = registration_fee - variable
net_fixed = max(0, fixed - sponsorship)

if contribution <= 0:
    print("Break-even is not possible at this registration fee")
else:
    # Ceiling division using integer arithmetic
    break_even = (net_fixed + contribution - 1) // contribution
    print("Minimum registrations to break even:", break_even)

total_cost = fixed + expected_people * variable
total_income = sponsorship + expected_people * registration_fee
print("Expected total cost: Rs.", total_cost)
print("Expected total income: Rs.", total_income)
print("Surplus/shortfall: Rs.", total_income - total_cost)
```

Sample output

```
Minimum registrations to break even: 60
Expected total cost: Rs. 42000
Expected total income: Rs. 40000
Surplus/shortfall: Rs. -2000
```

Note: The break-even calculation assumes every registered participant pays the registration fee and variable costs are incurred per participant.